Functional Programming in Lean

Prof. Dr. Peter Thiemann
Marius Weidner

University of Freiburg
Winter 2026

Sheet 3

Due: Monday, 2026-11-23

Important Information:

- Exercises are ungraded and do *not* need to be submitted.
- If you have questions, please post a message in the chat.
- The solutions will be discussed in the tutorial sessions.
- Start from the template `Template03.lean`.
- **This sheet covers two weeks, because there is no lecture on 2026-11-17. Week 1 covers Exercises 3.1, 3.2, and the maybe part of 3.3, week 2 the rest of 3.3 and Exercise 3.4. Week 1 is easier and week 2 is harder, so feel free to start with week 2 already in week 1.**

Exercise 3.1 (Overloading, week 1)

With type classes we can finally close the two gaps from the previous sheets.

1. On Sheet 1, a panicking `last'` was only possible for integer lists, because `panic!` needs a default value of the result type. The type class `Inhabited` provides this default value.

   ```
   class Inhabited (α : Type) where
     default : α
   ```

   Write a function `last'`, which returns the last element of a list and panics on the empty list.

   Examples:

   ```
   >>> last' [1, 2, 3]
   3
   >>> last' ([] : List Nat)
   PANIC at last': last' is undefined on the empty list
   0
   ```

2. On Sheet 2 you wrote `addV3`, `subV3`, and `negV3` for vectors. With instances of the type classes `Add`, `Sub`, and `Neg`, the usual operators `+`, `-`, and unary `-` work for vectors as well. The template defines `V3` as on Sheet 2.

   Write an `Inhabited` instance for `V3 α`, and `Add`, `Sub`, and `Neg` instances, which apply the operation to each coordinate.

   Examples:

```
>>> last' ([] : List (V3 Int))
PANIC at last': last' is undefined on the empty list
{ x := 0, y := 0, z := 0 }
>>> (⟨1, 2, 3⟩ + ⟨10, 20, 30⟩ : V3 Int)
{ x := 11, y := 22, z := 33 }
>>> (⟨1, 2, 3⟩ - ⟨10, 20, 30⟩ : V3 Int)
{ x := -9, y := -18, z := -27 }
>>> (-⟨1, -2, 3⟩ : V3 Int)
{ x := -1, y := 2, z := -3 }
```

Exercise 3.2 (Monoids, week 1)

A monoid is a set A with a binary operation $\otimes$ and a neutral element ϵ, such that $\otimes$ is associative and ϵ is neutral on both sides.

$$\forall x, y, z \in A.\ (x \otimes y) \otimes z = x \otimes (y \otimes z) \qquad \forall x \in A.\ \epsilon \otimes x = x = x \otimes \epsilon$$

As in the lecture, we capture monoids with a type class.

```
class Monoid (α : Type) where
  op : α → α → α
  id : α
```

The type class does not enforce the laws. The programmer has to make sure that every instance satisfies them. In Exercise 3.4 you prove that your instances do.

Always write `Monoid.op` and `Monoid.id` with the class name. A plain `id` refers to the identity function `fun x => x`.

1. Write `Monoid` instances for the following types.

   a) `Bool` with `&&` and `true`, named `boolAnd` like `MonoidInt` in the lecture.

   b) `Int` with `+` and `0`, named `intAdd`.

   c) `List` α with `++` and `[]`.

   d) `Option` α, where `Monoid.op` returns its first argument if that is `some`, and its second argument otherwise. The neutral element is `none`.

   e) $\alpha \times \beta$, where α and β have `Monoid` instances, with the operation applied to each component.

   f) $\alpha \to \beta$, where β has a `Monoid` instance, with the operation applied to the results of the two functions.

   Warning: For the instance for functions, introduce the type variables explicitly with `instance {α β : Type} [Monoid β] : Monoid (α → β)`. Otherwise Lean fails with the error `stuck at solving universe constraint`.

   Examples:

```
>>> Monoid.op true false
false
>>> Monoid.op (3 : Int) 4
7
>>> Monoid.op [1, 2] [3]
[1, 2, 3]
>>> Monoid.op (none : Option Nat) (some 1)
some 1
>>> Monoid.op (some 1) (some 2)
some 1
>>> Monoid.op (true, (100 : Int)) (false, 20)
(false, 120)
>>> (Monoid.op (· > 5) (· < 10) : Int → Bool) 7
true
```

2. Write a function `mconcat`, which combines all elements of a list with the operation of a monoid.

   ```
   def mconcat [m : Monoid α] : List α → α
   ```

   Examples:

   ```
   >>> mconcat ([1, 5, 2] : List Int)
   8
   >>> mconcat [true, false, true]
   false
   ```

3. Some types form a monoid with more than one operation. `Bool` is a monoid with `&&` and `true`, but also with `||` and `false`. `Int` is a monoid with `+` and `0`, but also with `*` and `1`. Write two more instances `boolOr` and `intMul` with low priority.

   ```
   instance (priority := low) boolOr : Monoid Bool where
   ```

   If several instances fit, Lean uses the one with the highest priority. So `boolAnd` and `intAdd` stay the default. Because the instance argument of `mconcat` has the name `m`, you can choose another instance by name, or provide a local instance with `let`.

   Examples:

   ```
   >>> mconcat [true, false, true]
   false
   >>> mconcat (m := boolOr) [true, false, true]
   true
   >>> mconcat (m := intMul) [1, 5, 2]
   10
   >>> let _ : Monoid Bool := boolOr; mconcat [true, false, true]
   true
   ```

A choice of instance at the call site does not show up in the types. When it has to, the usual workaround is a wrapper type with a single field, such as `structure BoolOr where val : Bool`, with its own instance.

Exercise 3.3 (Foldable, weeks 1 and 2)

In the lecture you saw `foldr` for lists. The type class `Foldable` generalizes it to other containers.

```
class Foldable (f : Type → Type) where
  foldr : (α → β → β) → β → f α → β
```

Here `f` is not a type but a function from types to types. If α is a type, then `f` α is a type. For example, `List` and `Option` have the type `Type → Type`, whereas `List Nat` and `Option Nat` are types.

The template defines the following binary tree.

```
inductive BTree (α : Type) where
  | leaf
  | node (l : BTree α) (x : α) (r : BTree α)

def exampleTree : BTree Int :=
  .node (.node .leaf 1 .leaf) 2
    (.node .leaf 3 (.node .leaf 4 .leaf))
```

1. Write `Foldable` instances for `List`, `Option`, and `BTree`. The fold over a tree visits the elements from left to right, i.e. first the left subtree, then the element of the node, then the right subtree.

   Hint: An instance cannot refer to itself. For `BTree`, first write a recursive function `BTree.foldr` and then use it in the instance.

   ```
   def BTree.foldr (f : α → β → β) (z : β) : BTree α → β
   ```

2. Write the following functions, which work for every foldable container.

   ```
   def fold [Foldable f] [Monoid α] (xs : f α) : α
   def toList [Foldable f] (xs : f α) : List α
   def length [Foldable f] (xs : f α) : Nat
   def sum [Foldable f] (xs : f Int) : Int
   ```

   `fold` combines all elements with the operation of the monoid, like `mconcat` in Exercise 3.2. Define `sum` with `fold`.

   Examples:

   ```
   >>> toList exampleTree
   [1, 2, 3, 4]
   >>> toList (some 5)
   [5]
   ```

```
>>> length exampleTree
4
>>> length [10, 20, 30]
3
>>> sum exampleTree
10
>>> fold [[1, 2], [3], [4, 5]]
[1, 2, 3, 4, 5]
```

Exercise 3.4 (Lawful monoids, week 2)

In the lecture, `LawfulBEq` adds the laws of equality to `BEq`. Its instances contain proofs. We do the same for monoids.

```
class LawfulMonoid (α : Type) [Monoid α] : Prop where
  op_assoc : ∀ x y z : α,
    Monoid.op (Monoid.op x y) z = Monoid.op x (Monoid.op y z)
  id_op : ∀ x : α, Monoid.op Monoid.id x = x
  op_id : ∀ x : α, Monoid.op x Monoid.id = x
```

The template contains a complete instance for `Nat` with `+` and `0` as an example. Put the cursor after `by` and on each tactic to see the current goal in the infoview. The worked example uses `omega`, which proves goals of linear arithmetic over `Nat` and `Int`. You may prove the laws with tactics, or without them by writing proof terms as in the lecture, e.g. with `fun`, `match`, and `rfl`. If you use tactics, the following ones are enough. The last column lists the items below in which our solution uses them.

Tactic	Effect	Items
`intro x`	names the variable of a goal `∀ x, ...`	2, 3, 4
`show e`	replaces the goal by the equal goal `e`	3, 4
`rfl`	proves a goal that holds by computation	2, 3
`decide`	checks a goal for all values, e.g. of `Bool`	1
`cases x with ...`	proves the goal for each constructor of `x`	2
`induction x with ...`	proves the goal by induction on `x`	3, 5
`simp only [h₁, h₂]`	rewrites with the equations h_1, h_2	3, 5
`rw [h]`	rewrites once with the equation `h`	4
`funext x`	proves `f = g` by proving `f x = g x`	4
`exact e`	proves the goal with the term `e`	4

If you write proof terms instead, the following building blocks are enough.

Term	Effect	Items
`fun x => e`	proves a goal `∀ x, ...`	1, 2, 3, 4
`match x with ...`	proves the goal for each constructor of `x`	1, 2
`rfl`	proves a goal that holds by computation	1, 2, 3
recursive theorem	proves the goal by induction	3, 5
`congrArg f h`	proves `f a = f b` from `h : a = b`	3, 5
`Prod.ext h₁ h₂`	proves `p = q` from `p.1 = q.1` and `p.2 = q.2`	4
`funext h`	proves `f = g` from `h : ∀ x, f x = g x`	4
`h.symm`	proves `b = a` from `h : a = b`	5
`h₁.trans h₂`	proves `a = c` from `a = b` and `b = c`	5

1. Prove the laws for your instance for `Bool`. *Hint:* `decide`.

2. Prove the laws for your instance for `Option` α. *Hint:* Split on the first argument with `cases`.

   ```
   cases x with
   | none => ...
   | some a => ...
   ```

3. Prove the laws for your instance for `List` α. *Hint:* After `show`, the goals are about `++`. Prove them by induction on the first list.

   ```
   intro xs ys zs
   show xs ++ ys ++ zs = xs ++ (ys ++ zs)
   induction xs with
   | nil => ...
   | cons x xs ih => ...
   ```

   In the case `cons`, the hypothesis `ih` is the law for the tail `xs`. The following equation from the standard library helps with `simp only`.

   ```
   List.cons_append : (x :: xs) ++ ys = x :: (xs ++ ys)
   ```

4. *Bonus:* Prove the laws for the instances for pairs and functions. Reuse the laws of the components with `rw` and `exact`.

5. *Bonus:* The laws are what make monoids useful. Prove that `mconcat` distributes over `++`.

   ```
   theorem mconcat_append [Monoid α] [LawfulMonoid α]
       (xs ys : List α) :
       mconcat (xs ++ ys) = Monoid.op (mconcat xs) (mconcat ys)
   ```

   Which laws does the proof use?